

นวัตกรรมเกษตรอัจฉริยะ: AIOT สำหรับเกษตรกรแม่นยำ รุ่นที่ 1

25 ก.ย. 2569 – 29 ก.ย. 2569

เครือข่ายไร้สายระยะไกล สำหรับเกษตรแปลงใหญ่

LoRa คืออะไร? (Introduction to LoRa)

LoRa (Long Range) คือ เทคโนโลยีการสื่อสารไร้สายที่ออกแบบมาสำหรับการส่งข้อมูลระยะไกล และใช้พลังงานต่ำมาก (Low-Power)

จุดเด่น: ถูกออกแบบมาเพื่อส่งข้อมูลขนาดเล็กเป็นระยะทางหลายกิโลเมตร โดยใช้พลังงานน้อยมาก

ข้อแตกต่างจาก Wi-Fi หรือ Bluetooth: LoRa ไม่ได้ถูกออกแบบมาเพื่อความเร็วสูง แต่เน้นความสำคัญในด้าน:

1. ระยะการสื่อสารที่ไกล (Long Communication Range)
2. การใช้พลังงานต่ำมาก (Low Power Consumption)
3. ความทนทานต่อสัญญาณรบกวนสูง (Robustness in Noisy Environments)
4. ในไทย ใช้ความถี่ 915 MHz

ตารางเปรียบเทียบ LoRa vs เทคโนโลยีไร้สายอื่นๆ

เทคโนโลยี	ระยะทาง (Range)	อัตราส่งข้อมูล (Data Rate)	การใช้พลังงาน (Power)	การใช้งานทั่วไป (Typical Use)
Wi-Fi	ใกล้	สูง	สูง	เข้าถึงอินเทอร์เน็ต
Bluetooth	ใกล้มาก	ปานกลาง	ต่ำ	อุปกรณ์สวมใส่
Cellular	ไกล	สูง	สูงมาก	ข้อมูลมือถือ
LoRa	ไกลมาก	ต่ำมาก	ต่ำมาก	เซนเซอร์ IoT

หมายเหตุ: LoRa ไม่ได้มาแทนที่ Wi-Fi หรือ Cellular แต่ถูกออกแบบมาสำหรับงาน Telemetry (ส่งค่าการวัด) ไม่ใช้การสตรีมข้อมูลต่อเนื่อง

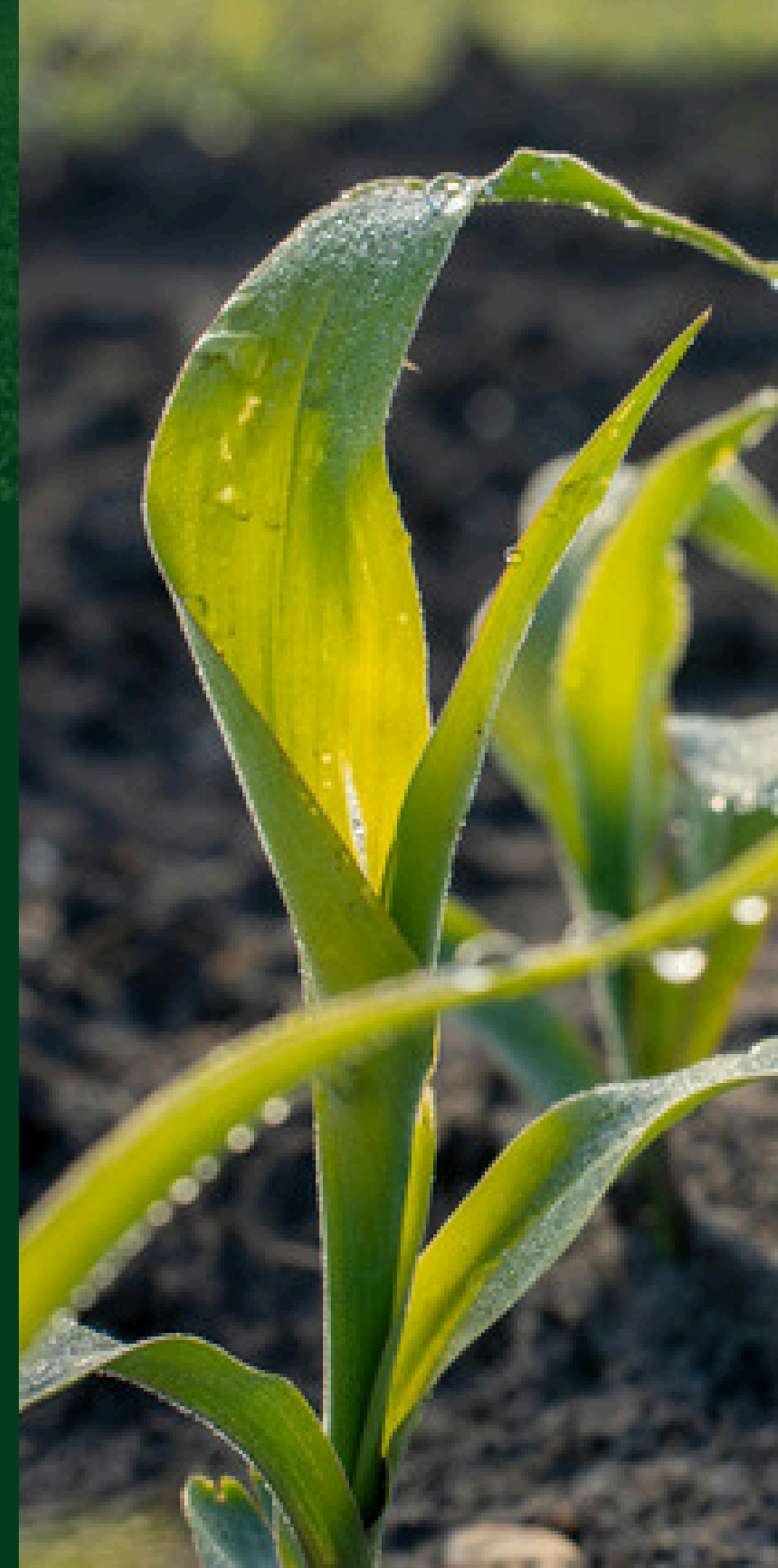
รูปแบบการใช้งาน LoRa

LoRaWAN

1. ใช้ Gateways และ Network Servers
2. บริหารจัดการด้วยมาตรฐานและระบบ Cloud Infrastructure
3. มีความยืดหยุ่นในระดับการควบคุมฮาร์ดแวร์ต่ำ (Low level) น้อยกว่า

Custom LoRa

1. ควบคุมการทำงานของคลื่นวิทยุโดยตรง
2. ไม่ต้องพึ่งพาระบบ Cloud
3. เข้าถึงรูปแบบแพ็คเกจ และจังหวะเวลาได้เต็มที่



ข้อจำกัดสำคัญของ LoRa (Limitations)

1. อัตราการส่งข้อมูลต่ำมาก (Very Low Data Rate)

1. ขนาดข้อมูล (Payload) ทัวไปอยู่เพียงแค่หลักสิบบิต
2. ไม่เหมาะสำหรับส่งข้อมูลเสียง รูปภาพ หรือวิดีโอ

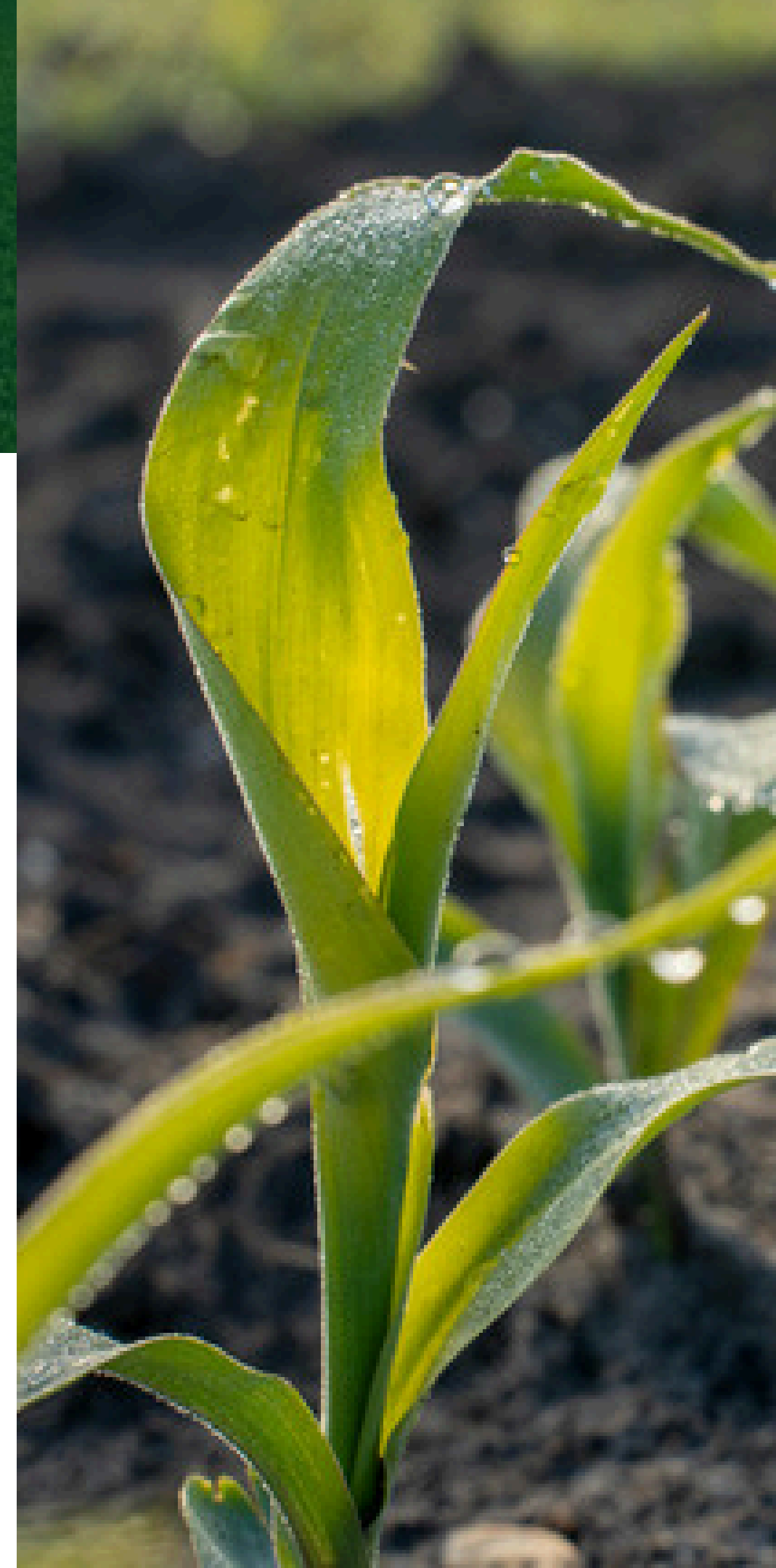
2. ความหน่วงสูง (High Latency)

การส่งข้อมูลใช้เวลาพอสมควร การตอบรับ (Acknowledgments) ช้าเมื่อเทียบกับ Wi-Fi

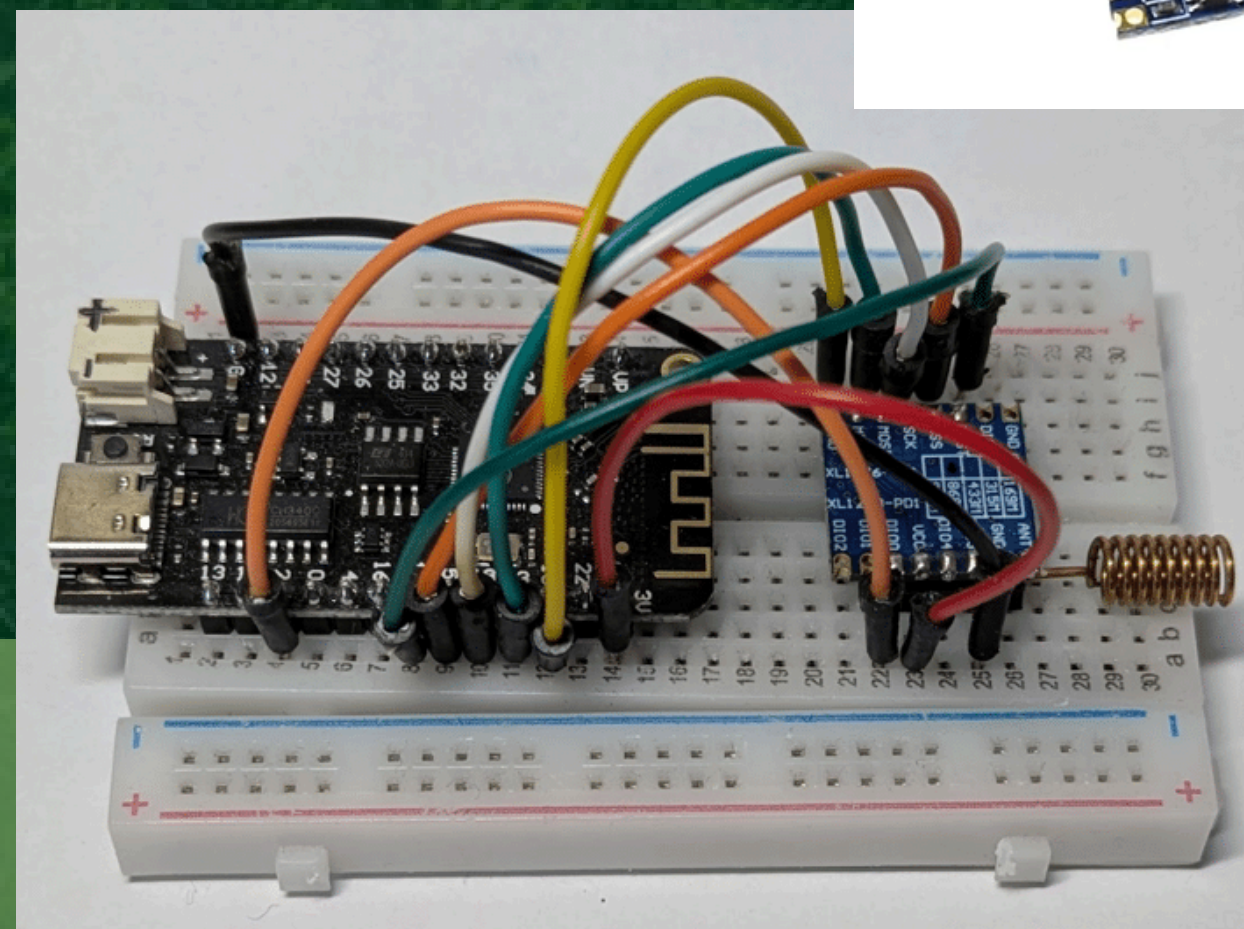
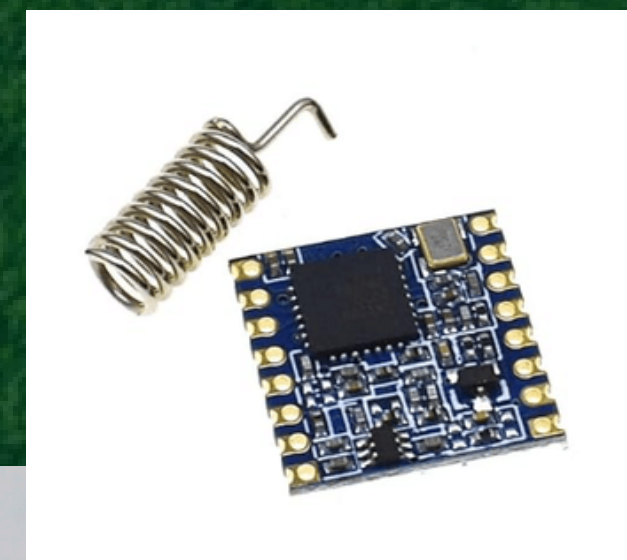
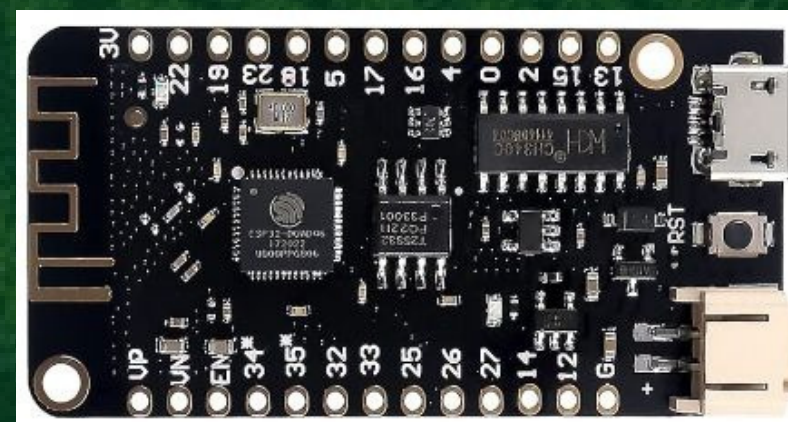
3. ข้อจำกัดเรื่อง Duty Cycle

มีข้อกำหนดทางกฎหมายจำกัดความถี่ในการส่งสัญญาณ ห้ามส่งสัญญาณแบบต่อเนื่องตลอดเวลา กฎหมายไทยให้แค่ 1%

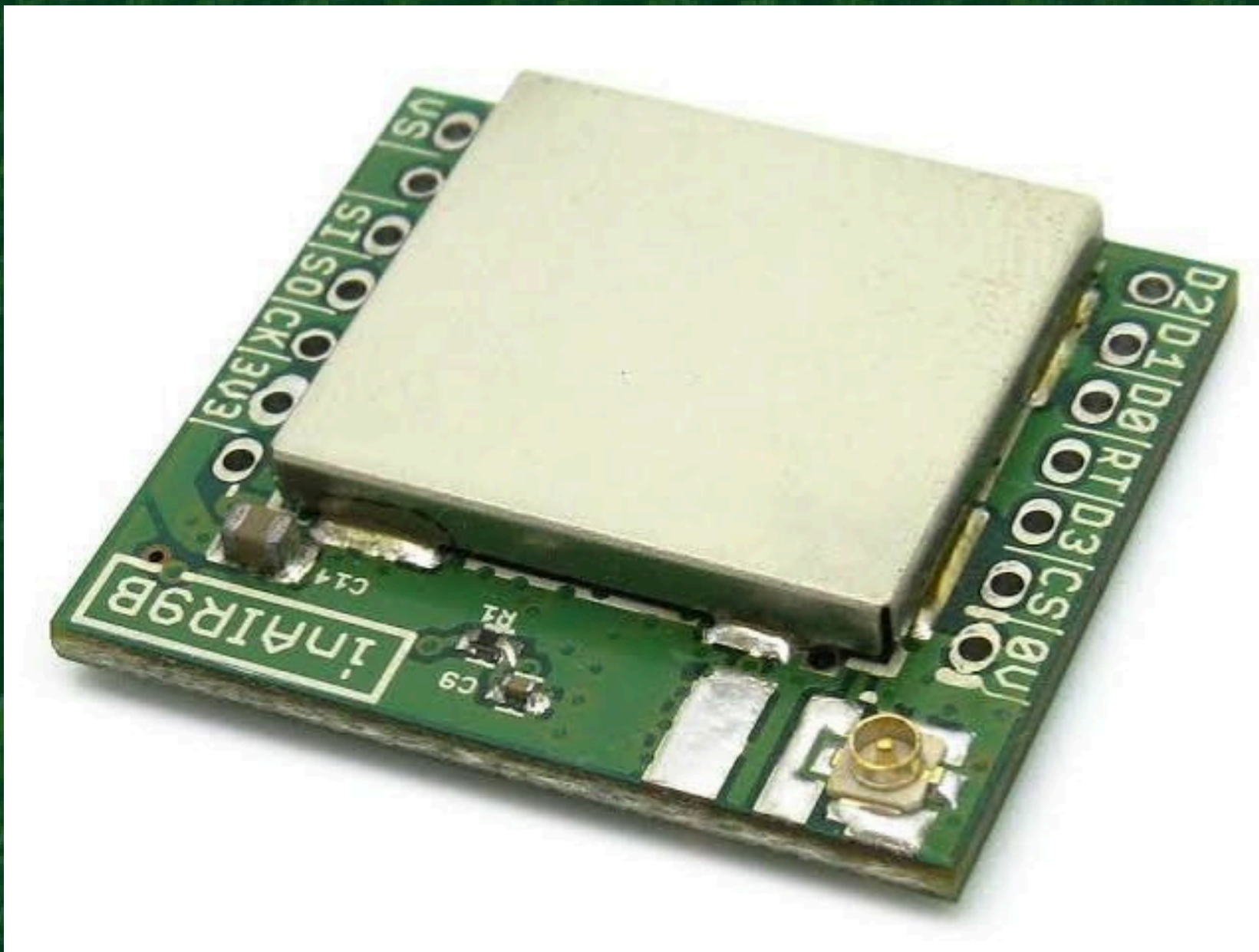
4. โมดูล SX1276 รองรับแรงดันไฟ 3.3 V เท่านั้น (ไม่ทนต่อไฟ 5 V การต่อไฟ 5 V จะทำให้โมดูลเสียหายถาวร)



มอดูโมดูล SX1276 เข้ากับบอร์ด ESP32 เอง ใช้ขาตามอินเตอร์เฟซ SPI หรือซื้อบอร์ดรวมสำเร็จ



เพิ่มระยะทาง ด้วยหัวต่อ n.fl กับสายแปลง n.fl เป็น sma เข้าสายอากาศเกน 8 dB



การกำหนดพิน (Pinout สำหรับ ESP32-C3)

ตัวอย่างการต่อสาย SPI สำหรับ ESP32-C3
เข้ากับ SX1276:

- **SCK:** GPIO 4
- **MISO:** GPIO 5
- **MOSI:** GPIO 6
- **NSS (CS):** GPIO 7
- **RST:** GPIO 10
- **DIO0:** GPIO 3
- **VCC:** 3.3V (⚠ ห้ามต่อ 5V) / **GND:** GND

ไลบรารี LoRa (sx1276.py หรือ lora.py)
เพื่อให้โค้ดหลักทำงานได้ บอร์ดทั้งสองต้องมี
ไฟล์ไลบรารี SX1276 สำหรับ MicroPython
(เช่น micropython-sx1276 หรือคลาส SX1276
พื้นฐาน) อยู่ในตัวบอร์ดก่อน

โค้ดสำหรับ บอร์ด 1 (Sender / Master)

หน้าที่: อ่านเวลาจากระบบ (time.ticks_ms() หรือ time.time()) ส่งไปให้บอร์ด 2 แล้วรอฟังค่า ACK ภายในช่วงเวลา timeout ถ้าได้ ACK จะคำนวณ Round-Trip Time (RTT) แล้วรอส่งรอบถัดไป

```
import time
from machine import Pin, SPI
from sx1276 import SX1276

# --- กำหนด Pinout สำหรับ ESP32-C3 ---
SCK_PIN = 4
MISO_PIN = 5
MOSI_PIN = 6
CS_PIN = 7
RST_PIN = 10
DIO0_PIN = 3

# ตั้งค่า SPI บน ESP32-C3
spi = SPI(1, baudrate=10000000, polarity=0, phase=0,
          sck=Pin(SCK_PIN), miso=Pin(MISO_PIN), mosi=Pin(MOSI_PIN))
```

```
# กำหนดพินสำหรับ SX1276
cs = Pin(CS_PIN, Pin.OUT)
rx_led = Pin(8, Pin.OUT) # (ตัวเลือก) พินไฟ LED ของ C3

# กำหนดค่า LoRa (ความถี่ 915MHz หรือ 868MHz ตามโมดูลที่ใช้)
lora = SX1276(spi, cs=cs, pins={'rst': RST_PIN, 'dio0': DIO0_PIN},
              bandwidth=125000, coding_rate=5, spreading_factor=7,
              preamble_length=8, frequency=915000000)

print("--- Board 1: Sender Ready ---")

msg_id = 0

while True:
    msg_id += 1
    send_time = time.ticks_ms()
    payload = f"TIME:{send_time}||ID:{msg_id}"
```

```
print(f"\n[Board 1] Sending: {payload}")
lora.println(payload) # ส่งแพ็คเกจ LoRa

# รอรับ ACK จาก Board 2 (Timeout 3000 ms)
ack_received = False
wait_start = time.time()

while time.time() - wait_start < 3000:
    if lora.available():
        try:
            packet_text = lora.read_payload().decode('utf-8')
            if packet_text == f"ACK:{msg_id}":
                rtt = time.time() - send_time
                rssi = lora.packet_rssi()
                print(f"[Board 1] Received ACK for ID {msg_id}! (RTT: {rtt} ms, RSSI: {rssi} dBm)")
                ack_received = True
                break
        except Exception as e:
            print("Error reading packet:", e)
    time.sleep_ms(20)
```

```
if not ack_received:  
    print(f"[Board 1] ACK Timeout for ID {msg_id}")  
  
# หน่วงเวลาก่อนส่งข้อความถัดไป (เช่น 2 วินาที)  
time.sleep(2)
```

โค้ดสำหรับ บอร์ด 2 (Receiver / Slave)

หน้าที่: รอฟังแพ็กเกจที่ขึ้นต้นด้วย TIME: เมื่อได้รับแล้วจะสกัดค่า ID ออกมา แล้วส่งข้อความตอบกลับในรูปแบบ ACK:<ID> ไปยังบอร์ด 1 กันที

```
import time
from machine import Pin, SPI
from sx1276 import SX1276

# --- กำหนด Pinout สำหรับ ESP32-C3 (ให้ตรงกับการต่อจริง) ---
SCK_PIN = 4
MISO_PIN = 5
MOSI_PIN = 6
CS_PIN = 7
RST_PIN = 10
DIO0_PIN = 3

# ตั้งค่า SPI บน ESP32-C3
spi = SPI(1, baudrate=10000000, polarity=0, phase=0,
          sck=Pin(SCK_PIN), miso=Pin(MISO_PIN), mosi=Pin(MOSI_PIN))
```

```
cs = Pin(CS_PIN, Pin.OUT)
```

```
# กำหนดค่า LoRa ให้ตรงกับ Board 1
```

```
lora = SX1276(spi, cs=cs, pins={'rst': RST_PIN, 'dio0': DIO0_PIN},  
             bandwidth=125000, coding_rate=5, spreading_factor=7,  
             preamble_length=8, frequency=915000000)
```

```
print("--- Board 2: Receiver Ready ---")
```

```
while True:
```

```
    if lora.received_packet():
```

```
        try:
```

```
            payload = lora.read_payload().decode('utf-8')
```

```
            rssi = lora.packet_rssi()
```

```
            snr = lora.packet_snr()
```

```
            print(f"\n[Board 2] Received: '{payload}' (RSSI: {rssi} dBm, SNR: {snr} dB)")
```

```
# ตรวจสอบรูปแบบข้อความ TIME:<time>||ID:<id>
if payload.startswith("TIME:"):
    parts = payload.split("|")
    time_val = parts[0].split(":")[1]
    msg_id = parts[1].split(":")[1]

# หน่วงเวลาเล็กน้อยให้ฝั่ง Sender สลับเข้าโหมดรับทัน
time.sleep_ms(100)

# ส่ง ACK กลับไป
ack_payload = f"ACK:{msg_id}"
print(f"[Board 2] Replying: {ack_payload}")
lora.println(ack_payload)

except Exception as e:
    print("Error processing packet:", e)

time.sleep_ms(20)
```